

- **LLM A:** Gemini
- **LLM B:** Chat Gpt

Tabela Comparativa de Modularização

Abaixo está o preenchimento da rubrica aplicando as notas **0** (não atende), **1** (atende parcialmente) ou **2** (atende plenamente):

#	Critério de modularização	LLM A	LLM B
1	Decomposição em funções, cada uma com responsabilidade única	2	1
2	Nomes descritivos de funções e variáveis	2	2
3	Ausência de duplicação de código (DRY)	2	1
4	Parâmetros e retornos bem definidos; evita variáveis globais	2	1
5	Reaproveitamento de funções (ex.: validação usada em um só lugar)	2	2
6	Existência de uma função principal organizando o fluxo	2	0
7	Legibilidade geral e comentários úteis (sem excesso)	2	2
	Total (Máximo de 14 pontos)	14	9

Justificativa Detalhada dos Resultados

1. Decomposição em funções com responsabilidade única

- **LLM A (Nota 2):** Excelente divisão de tarefas. Possui funções exclusivas para validar, converter unidades, calcular estatísticas e classificar estados de maneira isolada.

- **LLM B (Nota 1):** A função `processar_sensor` acumula responsabilidades excessivas. No mesmo bloco de código, ela filtra dados válidos, chama a classificação, faz varredura de sequências críticas e calcula métricas estatísticas (média, mínimo, máximo e desvio padrão).

2. Nomes descritivos de funções e variáveis

- **LLM A (Nota 2):** Variáveis extremamente claras e expressivas, como `maior_sequencia_critica`, `valores_convertidos` e `receber_medicoes_brutas`.
- **LLM B (Nota 2):** Abreviações limpas e fáceis de associar ao contexto do problema, como `rel_temp`, `vibracoes`, `alerta` e `critico`.

3. Ausência de duplicação de código (DRY)

- **LLM A (Nota 2):** Utiliza uma estrutura de dicionário (`CONFIG_SENSORES`) e um laço `for` dentro da função principal para processar dinamicamente qualquer sensor, eliminando repetições.
- **LLM B (Nota 1):** Apresenta repetição manual de blocos de código na execução final para extrair as listas (`temperaturas = [...]`, `pressoes = [...]`) e para invocar os relatórios de cada sensor individualmente em sequência.

4. Parâmetros/Retornos bem definidos e ausência de globais

- **LLM A (Nota 2):** Todas as funções recebem parâmetros explícitos e devolvem dicionários bem estruturados. Não há poluição do escopo global.
- **LLM B (Nota 1):** Por não possuir uma função isoladora de fluxo, o script cria e manipula diversas variáveis diretamente no escopo global do arquivo (`dados`, `temperaturas`, `rel_temp`, etc.), o que viola boas práticas de encapsulamento.

5. Reaproveitamento de funções

- **LLM A (Nota 2):** A função orquestradora `analisar_sensor` é integralmente reutilizada para todas as grandezas físicas configuradas no sistema.
- **LLM B (Nota 2):** As funções de validação, classificação e exibição de relatório são reaproveitadas com sucesso para os três sensores monitorados.

6. Existência de uma função principal organizando o fluxo

- **LLM A (Nota 2):** Implementa a função padrão `def main()`: acionada corretamente pela estrutura condicional `if __name__ == "__main__":`.
- **LLM B (Nota 0):** O fluxo de execução é solto e linear no final do arquivo, sem a presença de uma função principal estruturada.

7. Legibilidade geral e comentários úteis

- **LLM A (Nota 2):** Código muito bem documentado com *docstrings* nativos do Python nas funções e cabeçalhos divisores de módulos claros.
- **LLM B (Nota 2):** Estética visual excelente, com ótimo uso de espaçamentos em branco e divisores por linhas tracejadas que facilitam muito a leitura do script.